LAN CHAT AND FILE-SHARING APPLICATION

Using Python Socket Programming

ICS 214 - IT Workshop III

STUDENT	ROLL NUMBER	DEPARTMENT
Divya Ramteke	2025BCS0161	CSE

PROJECT TYPE	NETWORKING	SUBMISSION FILES
Python desktop application	TCP sockets + threading	IPYNB + PDF report

Abstract

This project is a local-network group chat application developed in Python. A central TCP server accepts multiple clients, forwards text messages, relays file data in chunks, and stores recent chat and file-sharing records in an SQLite database. A simple Tkinter interface lets users connect, chat, and select files. SHA-256 checks are performed by the server and receiving clients to detect incomplete or altered transfers. The project uses Python standard-library modules and does not require an external dataset or paid service.

Core idea

> One server manages the room; each client connects to it. Chat is broadcast to the room, and a shared file is relayed to the other connected users.

Project scope: a course prototype for trusted LAN testing. It is not intended as a secure public messaging service.

1. Problem Statement and Objectives

Students on the same local network may need a simple way to exchange messages and files without depending on an external chat service. The project demonstrates how client-server networking can be implemented with Python and how multiple users can communicate through one server.

Objectives

• Implement client-server communication with TCP sockets.

• Allow multiple clients to join one chat room and exchange messages in real time.

• Provide basic file sharing with chunked transmission and integrity checks.

• Handle concurrent users using threads.

• Save chat history locally with SQLite and display recent history to new users.

• Provide a simple Tkinter GUI for connecting, chatting, and choosing files.

2. System Architecture

SERVER PROCESS
TCP listener on port 5000 | client registry | message broadcast | file relay | SQLite history
COMMUNICATION LAYER
Length-prefixed packets: JSON header + optional binary payload. File bytes are transferred in 64 KB chunks.
CLIENT APPLICATIONS
Tkinter GUI | username and server IP | chat view | send button | file picker | receiving-file verification

Key modules

Module	Responsibility
server.py	Accepts connections, checks usernames, broadcasts chat, relays file chunks, stores history, and enforces configured limits.
client.py	Connects to the server, shows messages in Tkinter, sends chat, shares files, and verifies received bytes.
SQLite database	Created automatically as chat_history.db beside server.py; stores message/file-event history.

3. Functionalities and Implementation

Functionality	Implementation
Multi-user chat	A thread handles each client connection; text messages are broadcast to registered clients.
File sharing	The client sends file metadata then 64 KB chunks. The server relays them to the other connected clients.
TCP sockets	A TCP connection carries length-prefixed metadata and optional binary data, preserving packet boundaries in the application protocol.
Multithreading	Concurrent server threads and a background file-sending thread keep message handling responsive.
Tkinter interface	Connection fields, chat history, message input, Send button, Share File button, and a status line.
SQLite history	Chat messages and successfully shared-file records are stored in chat_history.db; new users can receive up to 100 recent records.
SHA-256 integrity	The server checks received byte count and digest. Each recipient independently checks the received file before keeping it.
Error handling	The implementation checks message sizes, file size, duplicate usernames, failed connections, missing transfer records, and incomplete or corrupted transfers.

4. Test Plan and Results

Test	Observed result
Syntax compilation of both scripts	Passed.
Two local clients exchange a chat message	Passed in a local protocol smoke test.
Binary file transfer (115,333 bytes)	Passed; recipient reconstructed identical bytes and SHA-256 matched.
Client disconnect	Disconnect path was exercised in a local protocol test.
Two Tkinter clients exchange chat and a file	Passed under a virtual display; received file contents and SHA-256 verified.
100 simulated local clients connect	Passed connection-capacity check; 101st connection was rejected by the configured limit.
100 users on a physical Wi-Fi router	Not benchmarked. Real-network performance remains to be tested in the intended environment.

5. Run Instructions

Local test on one computer

• Open a terminal in the project folder and run: python server.py

• Open a second terminal and run: python client.py

• Keep Server IP as 127.0.0.1, enter a username, and click Connect.

• Run client.py in a third terminal for a second user, using a different username.

• Send a message. Use Share File to select a file; other connected clients receive it under received_files/.

Test on the same Wi-Fi/LAN

• Run server.py on the computer that will host the chat.

• Find its local IPv4 address (on Windows, use ipconfig).

• On each client computer, run client.py and enter the server computer IP and a unique username.

• Allow Python on a private network in the firewall if prompted. Some guest or campus networks block device-to-device traffic.

Commands

```
python server.py
python client.py
```

6. Limitations and Responsible Use

• The default registered-client cap is 100. The local test confirmed 100 simulated connections, but did not benchmark 100 people actively chatting or transferring files on a real router.

• Maximum file size is 25 MB. File sharing broadcasts to all other connected users and does not support recipient selection in this version.

• The prototype has no account authentication or message encryption. Use only on a trusted local test network; do not send sensitive information or expose port 5000 to the public internet.

• Guest Wi-Fi client isolation, firewall rules, router limits, and the server computer can affect cross-device connections and performance.

7. Conclusion

The project provides a practical demonstration of TCP socket programming, threading, desktop UI development, file transfer, database storage, and file-integrity checking. The two main Python programs keep the runnable application structure compact, while the Jupyter Notebook documents the implementation and the PDF records the design and validation. The next step is to run the GUI on the target computer and conduct the live multi-client demonstration over the intended LAN.

Deliverables: server.py, client.py, LAN_Chat_Project.ipynb, Project_Report.pdf, and README.md.